Project Mockingbird

Synthetic Security Log Generation with CTGAN
for Microsoft Sentinel Detection Testing

Hamad Muaid

October 2026

Contents

1 Introduction **2**
- 1.1 Success criteria . . . 2
- 1.2 Project workflow . . . 2

2 Environment Setup **3**
- 2.1 Environment overview . . . 3
- 2.2 Installation and dataset transfer . . . 3

3 Data Profiling **4**
- 3.1 Dataset overview . . . 4
- 3.2 Profiling results . . . 4

4 Preprocessing **6**
- 4.1 Purpose and steps . . . 6
- 4.2 Implementation and output . . . 6
- 4.3 Verification . . . 7
- 4.4 Issue encountered . . . 7

5 CTGAN Training **8**
- 5.1 Model inputs . . . 8
- 5.2 Configuration . . . 8
- 5.3 Training runs . . . 8
- 5.4 Final training run . . . 8

6 Synthetic Data Generation **9**
- 6.1 Generation process . . . 9
- 6.2 Deriving Task . . . 9
- 6.3 Script versions and output . . . 9

7 Post-processing **12**
- 7.1 Purpose and steps . . . 12
- 7.2 Output . . . 12
- 7.3 Format validation . . . 13

8 Quality Evaluation **14**
- 8.1 Method . . . 14
- 8.2 Tuning progression . . . 14
- 8.3 Final result . . . 17

9 Sentinel Integration **18**
- 9.1 Overview . . . 18
- 9.2 Storage account and SFTP upload . . . 18
- 9.3 Custom table and data collection rule . . . 20
- 9.4 Ingestion script . . . 21
- 9.5 Verification . . . 22

10 Validation **23**
- 10.1 Approach . . . 23
- 10.2 Password spray detection . . . 23
- 10.3 Results . . . 24

11 Conclusion **25**

1 Introduction

Testing detection rules in a Security Information and Event Management (SIEM) system such as Microsoft Sentinel needs realistic log data. Using real production logs for this risks exposing personally identifiable information (PII) and breaching data privacy policy.

This project uses a **Conditional Tabular Generative Adversarial Network (CTGAN)**.

- The model trains on sample Microsoft Sentinel Logs and generates thousands of synthetic logs that match the same patterns.
- (SOC) team can use these logs to test and validate detection logic without handling real data

The project is a machine learning initiative to strengthen Cyber Defence testing. This report documents the full pipeline, from environment setup and data preparation to model training, quality evaluation and delivery to Sentinel.

1.1 Success criteria

The project is measured against four success criteria. Table **??** lists each one with its current status.

Category	Measurement of success	Status
Data fidelity	SDMetrics quality score of at least 80% (column shapes and pair trends compared with the real data)	Met (83.8%)
Formatting	100% of generated IP addresses, timestamps and usernames match the structural format of the original dataset	Pending
Integration	Synthetic `.csv` ingested into a Sentinel custom log workspace (`_CL`) without parsing errors	Pending
Validation	SOC team triggers at least one existing SIEM analytic rule using the synthetic log data	Pending

1.2 Project workflow

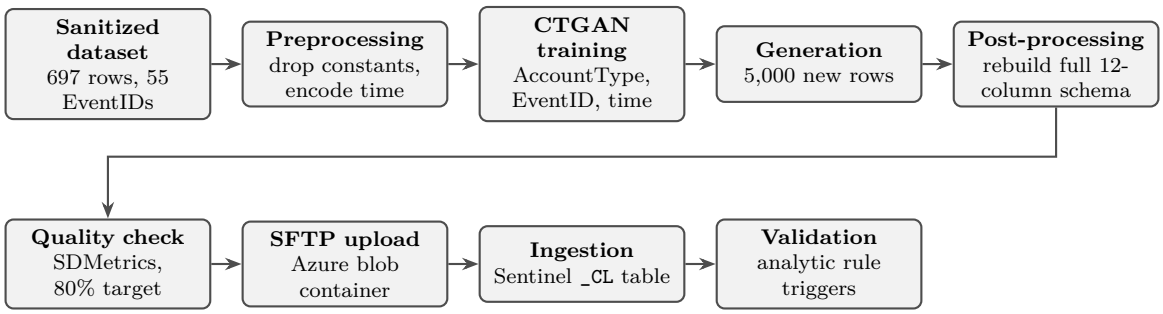


Figure 1: Project Mockingbird workflow, from the sanitized dataset -> detection-rule validation.

2 Environment Setup

2.1 Environment overview

- Pipeline runs on an Ubuntu Desktop 26.04 virtual machine in VMware
- Linux was chosen because the machine learning libraries install more cleanly there than on Windows
- Used a virtual environment, `mockingbird-env`, to keep the projects packages separate from the system python

2.2 Installation and dataset transfer

The commands below set up the environment. **PyTorch** is installed in its CPU-only build because the virtual machine has no GPU. **CTGAN** is installed without its dependencies so that it does not replace that build; the missing **RDT** package is then added separately.

```
# System packages
sudo apt update
sudo apt install -y python3-pip python3-venv

# Virtual environment
python3 -m venv mockingbird-env
source mockingbird-env/bin/activate

# CPU-only PyTorch (avoids multi-GB CUDA packages)
pip install torch --index-url https://download.pytorch.org/whl/cpu --no-cache-dir

# CTGAN without its default dependency pull
pip install ctgan --no-cache-dir --no-deps

# Remaining libraries
pip install sdmetrics pandas openpyxl
pip install "rdt>=1.20.0" --no-cache-dir
pip install sdv --no-cache-dir
```

Table 1 lists the libraries and their roles:

Library	Purpose
PyTorch (CPU)	Deep learning engine used by CTGAN
CTGAN	Generative model for tabular data
RDT	Encodes and decodes columns for CTGAN
SDMetrics	Quality score for the synthetic data
SDV	Metadata helper used by the evaluation script
pandas	Loading and transforming the datasets
openpyxl	Reading the `.xlsx` dataset

Table 1: Python libraries used in the project.

- The training dataset was provided internally and is already sanitized.

```
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~/mockingbird$ ls -l
total 52
-rw-rw-r-- 1 hamad hamad 50790 Sep 16 12:57 Sentinel_Training_DataSet.xlsx
```

Figure 2: Sanitized training dataset present on the Ubuntu virtual machine.

3 Data Profiling

3.1 Dataset overview

The training dataset is a sanitized sample of Windows security events in the Sentinel `SecurityEvent` format.

- It has 697 rows and 12 columns
- 55 distinct EventIDs with about 15 rows each
- Profiling is done first to identify which columns the model can learn froma and which should be separated

3.2 Profiling results

Figure 3 shows the column types, missing values and unique-value counts.

```
Shape: (697, 12)

TenantId            object
TimeGenerated       object
SourceSystem        object
Account             object
AccountType         object
Computer            object
EventSourceName     object
Channel             object
Task                 int64
Level              float64
EventID              int64
Activity            object
dtype: object

Nulls:
TenantId             0
TimeGenerated        0
SourceSystem       489
Account              0
AccountType        274
Computer             0
EventSourceName      0
Channel              0
Task                 0
Level              489
EventID              0
Activity             0
dtype: int64

Unique values:
TenantId             1
TimeGenerated      398
SourceSystem         1
Account            697
AccountType          2
Computer           697
EventSourceName      1
Channel              1
Task                31
Level                1
EventID             55
Activity            48
dtype: int64
```

Figure 3: Dataset profile: shape, data types, null counts and unique values per column.

Results:

- Five columns are constant -> so there is nothing for the model to learn.
- **`Account`** and **`Computer`** are unique in every row, which a GAN cannot model realistically.
- **`AccountType`** has 274 missing values. **`SourceSystem`** and `Level` are empty in 489 of the 697 rows, but the synthetic logs fill them on every row.

Table 2 gives the role of each column and how it is handled.

Column	Role	Handling
`TenantId`, `SourceSystem`, `EventSourceName`, `Channel`, `Level`	Constant	Set aside and reattached after generation
`Account`, `Computer`	Unique in every row	Resampled from the real values
`AccountType`	Categorical, 274 missing	Modeled: Replaced all null values with 'Unknown' so the model can learn and generate accurate output
`EventID`	Categorical, 55 values	Modeled
`Task`	Categorical, follows EventID	Rebuilt from EventID
`Activity`	Description text, one per EventID	Rebuilt from EventID
`TimeGenerated`	Timestamp	Modeled: Converted to Unix time

Table 2: Column roles and handling.

4 Preprocessing

4.1 Purpose and steps

Preprocessing reduces the dataset to the columns CTGAN can learn from and puts them in a form the model accepts. The script `preprocess.py` is available in the project repository (https://github.com/upny-hm/mockingbird/blob/main/src/preprocess.py).
It performs five steps:

1. **Save the constants:** The five constant columns are stored so they can be reattached after generation.
2. **Drop unmodeled columns:** The constants and `Activity` field are removed, since the model cannot learn from constants and `Activity` is rebuilt from `EventID` later.
3. **Fill missing account types:** Missing `AccountType` values become "Unknown", which keeps every row and turns the gap into a category the model can learn.
4. **Parse timestamps:** `TimeGenerated` is read with the ISO 8601 format, because the original timestamps have mixed fractional seconds.
5. **Convert time to a number:** The timestamp is converted to Unix seconds, giving CTGAN a plain numeric column.

4.2 Implementation and output

Figure 4 shows its output -> The result is saved as `preprocessed_variable.csv`.

```
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~$ python3 preprocess.py
Constants saved: {'TenantId': '6618e8e6-0ec0-4001-9280-a9835bdbd933', 'SourceSystem': 'OpsManager', 'EventSourceName': '
Microsoft-Windows-Security-Auditing', 'Channel': 'Security', 'Level': np.float64(0.0)}

Remaining columns: ['Account', 'AccountType', 'Computer', 'Task', 'EventID', 'TimeGenerated_unix']

Shape: (697, 6)

Sample:
     Account AccountType     Computer   Task  EventID  TimeGenerated_unix
0  Account1     Machine  10.168.5.16  12289     4611          1789473926
1  Account2     Machine  10.168.5.17  12289     4611          1789473924
2  Account3     Machine  10.168.5.18  12289     4611          1789473921
3  Account4     Machine  10.168.5.19  12289     4611          1789473916
4  Account5     Machine  10.168.5.20  12289     4611          1789473905

Saved preprocessed_variable.csv
```

Figure 4: Preprocessing output: constants saved and the variable columns written to CSV.

4.3 Verification

A null check on the result (Figure 5) confirms that no missing values remain.

```
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~$ python3 -c "
import pandas as pd
df = pd.read_csv('preprocessed_variable.csv')
print(df.isnull().sum())
"
Account              0
AccountType          0
Computer             0
Task                 0
EventID              0
TimeGenerated_unix   0
dtype: int64
```

Figure 5: Null check on the preprocessed dataset.

4.4 Issue encountered

The first run failed while parsing `TimeGenerated`, because some timestamps have fractional seconds and others do not. Parsing with `format='ISO8601'` fixed it.

5 CTGAN Training

5.1 Model inputs

Only the columns with learnable patterns are passed to the model (Table 3). `Account`, `Computer` and `Task` are excluded for the reasons given in Section 3.

Column	Type
`AccountType`	Categorical
`EventID`	Categorical (cast to string)
`TimeGenerated_unix`	Continuous

Table 3: Columns modeled by CTGAN.

5.2 Configuration

CTGAN's default batch size of 500 is almost the size of the whole 697-row dataset, so each epoch makes about one update.

- The final model was trained @ Epochs= 800 and Batch size= 50

With a batch size of 50, 800 epochs give about 10,400 updates instead of 300, which gave the model far more chances to learn. The training script `train_ctgan.py` is available in the project repository (https://github.com/upny-hm/mockingbird/blob/main/src/train_ctgan.py), and the trained model is saved as `ctgan_model.pkl`.

5.3 Training runs

The model was trained five times, changing one thing each run.The table below lists the changes and the resulting overall quality score. The scoring method is explained in **Section** 8.

Run	Change	Epochs / batch	Score
1	All six columns, default settings	300 / 500	61.6%
2	`Account` and `Computer` removed	300 / 500	68.1%
3	`Task` removed	300 / 500	71.7%
4	More epochs, smaller batches	500 / 70	78.4%
5	More epochs, smaller batches	800 / 50	83.8%

5.4 Final training run

- Final training run produced an overall quality score of 84%
- For further information about specific training runs see: **Section** 8

6 Synthetic Data Generation

6.1 Generation process

The script `generate_synthetic.py` is available in the project repository (https://github.com/upny-hm/mockingbird/blob/main/src/generate_synthetic.py). It runs four steps:

- **Sample:** load the saved model and generate 5,000 rows of `AccountType`, `EventID` and time.
- **Resample identities:** fill `Account` and `Computer` with random values from the real data, so every username and hostname is valid.
- **Fix types:** convert `EventID` back to an integer.
- **Derive Task:** look up `Task` from the generated `EventID` (Section 6.2).

6.2 Deriving Task

- `Task` is determined by `EventID` , so it is looked up instead of generated.
- Some EventIDs map to more than one Task. Taking the most common one (the mode) dropped the rarer Task entirely.
- The final version **draws a Task at random from the values seen for that EventID in the real data, so rare Tasks keep their real frequency.**
- The script prints the number of unmapped Tasks, which is 0.

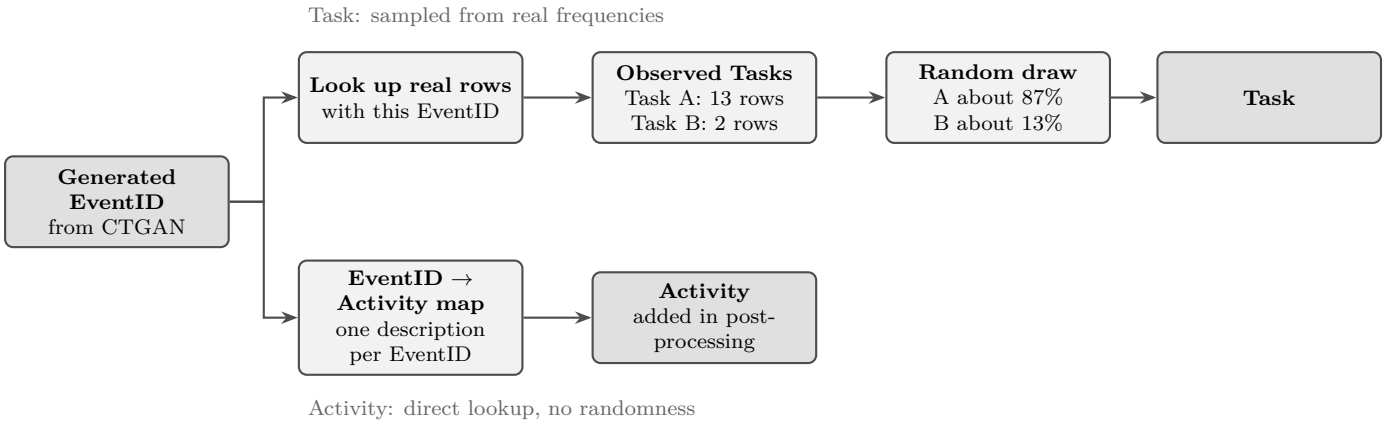


Figure 6: How `Task` and `Activity` are derived from the generated `EventID`.

6.3 Script versions and output

- **First version:** samples all rows straight from the model (Figure 7). The output has the expected six fields (Figure 8).
- **Validity check:** every categorical value in the synthetic data also exists in the real data, with 0 unseen values (Figure 9).
- **Second version:** `Account` and `Computer` are resampled from the real pool (Figure 10). The run produces 5,000 rows with 6 columns (Figure 11).
- **Final version:** adds the probabilistic Task lookup (Figure 12).

```
 GNU nano 8.7.1                                                generate_synthetic.py
# Hamad Musaid - Project Mockingbird

import pandas as pd
from ctgan import CTGAN

# Load trained model
ctgan = CTGAN.load('ctgan_model.pkl')

# Generate synthetic rows (5000)
n_samples = 5000
synthetic_data = ctgan.sample(n_samples)

print("Synthetic data shape:", synthetic_data.shape)
print("\nSample rows:\n", synthetic_data.head())

synthetic_data.to_csv('synthetic_variable.csv', index=False)
print("\nSaved synthetic_variable.csv")
```

Figure 7: First version of the generation script.

```
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~$ python3 -c "
import pandas as pd
df = pd.read_csv('synthetic_variable.csv')
print('Fields:', list(df.columns))
print()
print(df.head(10).to_string())
"
Fields: ['Account', 'AccountType', 'Computer', 'Task', 'EventID', 'TimeGenerated_unix']

      Account AccountType      Computer   Task  EventID  TimeGenerated_unix
0   Account41        User   10.235.9.42  12544     4797          1789476521
1  Account398        User   10.24.2.216  12810     4798          1789487234
2  Account171     Unknown   10.235.9.47  12544     4769          1789489725
3  Account580        User   10.24.2.131  12554     4728          1789469795
4   Account42     Unknown  10.168.5.125  12812     4662          1789486182
5  Account590     Machine   10.235.9.48  12810     4768          1789485235
6  Account194     Unknown   10.24.2.198  14337     4799          1789480769
7  Account499     Machine   10.24.2.198  12812     4770          1789491967
8  Account591     Unknown   10.24.2.141  14339     4663          1789480974
9  Account651     Unknown  10.168.5.197  12804     5059          1789469818
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~$
```

Figure 8: Synthetic data fields and sample rows from the first version.

```
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~$ python3 -c "
import pandas as pd
real = pd.read_csv('preprocessed_variable.csv')
synth = pd.read_csv('synthetic_variable.csv')

for col in ['Account','AccountType','Computer','Task','EventID']:
    real_vals = set(real[col].astype(str))
    synth_vals = set(synth[col].astype(str))
    unseen = synth_vals - real_vals
    print(f'{col}: {len(unseen)} unseen values out of {len(synth_vals)} unique synthetic values')
"
Account: 0 unseen values out of 696 unique synthetic values
AccountType: 0 unseen values out of 3 unique synthetic values
Computer: 0 unseen values out of 695 unique synthetic values
Task: 0 unseen values out of 31 unique synthetic values
```

Figure 9: Check for synthetic values that do not exist in the real data.

```
 GNU nano 8.7.1                                                    generate_synthetic.py *
# Hamad Musaid - Project Mockingbird
import pandas as pd
from ctgan import CTGAN

real = pd.read_csv('preprocessed_variable.csv')
ctgan = CTGAN.load('ctgan_model.pkl')

n_samples = 5000
synthetic_data = ctgan.sample(n_samples)

# Resample Account/Computer independently from the real pool
synthetic_data['Account'] = real['Account'].sample(n=n_samples, replace=True).values
synthetic_data['Computer'] = real['Computer'].sample(n=n_samples, replace=True).values

print("Synthetic data shape:", synthetic_data.shape)
print("\nSample rows:\n", synthetic_data.head())

synthetic_data.to_csv('synthetic_variable.csv', index=False)
print("\nSaved synthetic_variable.csv")
```

Figure 10: Second version: `Account` and `Computer` resampled from the real pool.

```
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~$ python3 generate_synthetic.py
Synthetic data shape: (5000, 6)

Sample rows:
   AccountType   Task EventID  TimeGenerated_unix     Account      Computer
0      Machine  13824    5061          1789461646  Account102   10.24.2.230
1      Unknown  13571    4798          1788649274  Account405   10.24.2.225
2      Machine  12811    4768          1789465433  Account464  10.235.9.144
3         User  12545    6427          1789464073  Account460  10.235.9.146
4         User  12810    4800          1789295707  Account167   10.168.5.68

Saved synthetic_variable.csv
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~$
```

Figure 11: Run of the second version: 5,000 rows, 6 columns.

```
 GNU nano 8.7.1                                                    generate_synthetic.py *
# Hamad Musaid - Project Mockingbird
import pandas as pd
from ctgan import CTGAN

real = pd.read_csv('preprocessed_variable.csv')
ctgan = CTGAN.load('ctgan_model.pkl')

n_samples = 5000
synthetic_data = ctgan.sample(n_samples)

# Resample Account/Computer independently from the real pool
synthetic_data['Account'] = real['Account'].sample(n=n_samples, replace=True).values
synthetic_data['Computer'] = real['Computer'].sample(n=n_samples, replace=True).values

# Deriving Tsk from EventID - Most common EventID in real Data
# (preserves rare secondary Task values instead of collapsing to the mode)
def sample_task(event_id, real_df):
    options = real_df.loc[real_df['EventID'] == event_id, 'Task']
    if len(options) == 0:
        return None
    return options.sample(n=1, weights=None).values[0]  # uniform draw from that EventID's actual observed Task values (preserves frequency since options has duplicates)

synthetic_data['EventID'] = synthetic_data['EventID'].astype(int)
synthetic_data['Task'] = synthetic_data['EventID'].apply(lambda eid: sample_task(eid, real))


print("Synthetic data shape:", synthetic_data.shape)
print("\nSample rows:\n", synthetic_data.head())
print("\nUnmapped Task:", synthetic_data['Task'].isnull().sum())

synthetic_data.to_csv('synthetic_variable.csv', index=False)
print("\nSaved synthetic_variable.csv")
```

Figure 12: Final script with probabilistic Task derivation.

7 Post-processing

7.1 Purpose and steps

The model output has only six columns, while Sentinel expects the full 12-column `SecurityEvent` schema. The script `postprocess.py` (`https://github.com/upny-hm/mockingbird/blob/main/src/postprocess.py`) rebuilds it in five steps:

- **Build the lookup:** map each `EventID -> Activity` using the original dataset.
- **Add Activity:** fill `Activity` on every row, with 0 unmapped.
- **Reattach constants:** restore `TenantId`, `SourceSystem`, `EventSourceName`, `Channel` and `Level`.
- **Restore timestamps:** convert Unix time back to the Sentinel format `YYYY-MM-DDTHH:mm:ss.fffZ`.
- **Reorder columns:** match the original schema and save `synthetic_final.csv`.

`Activity` is copied exactly as it appears in the original data, so a few EventIDs keep their original placeholder text (such as "-").

7.2 Output

- The result is 5,000 rows by 12 columns (about 1 MB).
- Figure 13 shows the script, Figure 14 shows the run, and Figure 15 shows a sample of the final file.

```
 GNU nano 8.7.1                                                    postprocess.py
# Hamad Musaid - Project Mockingbird

import pandas as pd

# Load original data (for EventID -> Activity mapping) and synthetic data
original = pd.read_excel('Sentinel_Training_DataSet.xlsx', sheet_name='Sheet1')
synth = pd.read_csv('synthetic_variable.csv')

# Build EventID -> Activity lookup from original data
event_activity_map = original.drop_duplicates('EventID').set_index('EventID')['Activity'].to_dict()

# Reattach constant columns
synth['TenantId'] = '6618e8e6-0ec0-4001-9280-a9835bdbd933'
synth['SourceSystem'] = 'OpsManager'
synth['EventSourceName'] = 'Microsoft-Windows-Security-Auditing'
synth['Channel'] = 'Security'
synth['Level'] = 0.0

# Rebuild Activity from EventID
synth['Activity'] = synth['EventID'].map(event_activity_map)

# Convert TimeGenerated_unix back to Sentinel's ISO8601 string format
synth['TimeGenerated'] = pd.to_datetime(synth['TimeGenerated_unix'], unit='s').dt.strftime('%Y-%m-%dT%H:%M:%S.0000000Z')
synth = synth.drop(columns=['TimeGenerated_unix'])

# Reorder columns to match the original schema
final_cols = [c for c in original.columns if c in synth.columns]
synth_final = synth[final_cols]

print("Final columns:", list(synth_final.columns))
print("Shape:", synth_final.shape)
print("\nUnmapped Activity (should be 0):", synth_final['Activity'].isnull().sum())
print("\nSample:\n", synth_final.head())

synth_final.to_csv('synthetic_final.csv', index=False)
print("\nSaved synthetic_final.csv")
```

Figure 13: The post-processing script.



Figure 14: Post-processing run: 12 columns, 5,000 rows, 0 unmapped Activity.

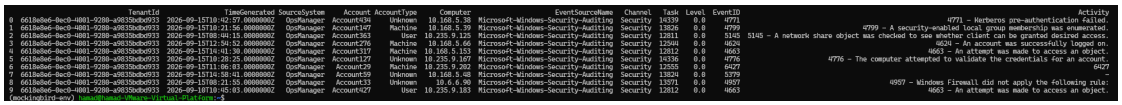


Figure 15: Sample rows from `synthetic_final.csv`.

7.3 Format validation

- A regular-expression check on `synthetic_final.csv` tests the three formats named in the success criteria: IP addresses (`Computer`), usernames (`Account`) and timestamps (`TimeGenerated`).
- The check is implemented through the script `validate_format.py` (`https://github.com/upny-hm/mockingbird/blob/main/src/validate_format.py`).
- It reports the share of rows matching each pattern, for both the original and the synthetic data. The target is 100%.

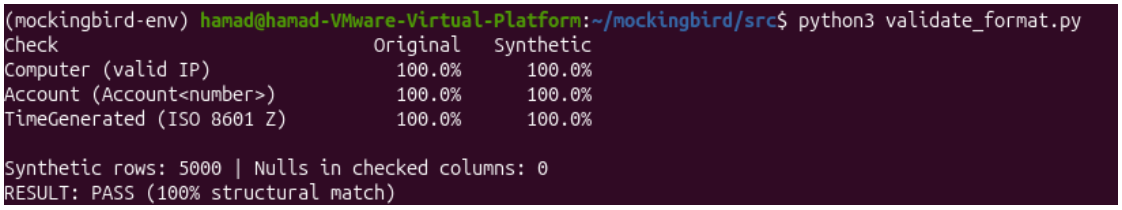


Figure 16: Format validation of IP addresses, usernames and timestamps.

8 Quality Evaluation

8.1 Method

- The SDMetrics `QualityReport` compares the synthetic data with the real data. The script `evaluate_quality.py` is available in the project repository (https://github.com/upny-hm/mockingbird/blob/main/src/evaluate_quality.py).
- **Column Shapes:** does each column's distribution match the real one?
- **Column Pair Trends:** do the relationships between pairs of columns match?
- **Overall score:** the average of the two -> The target is 80%.
- **NOTE:** The final version scores only the modeled columns (`AccountType`, `EventID` and time), because the other columns are either resampled or derived.

8.2 Tuning progression

- The model was tuned over five runs (Table 4). Figure 17 summarizes the scores, and Figures 18 to 25 show the evidence for each run.
- Runs 2 and 3 score fewer columns than run 1, so only runs 3 to 5 are directly comparable.

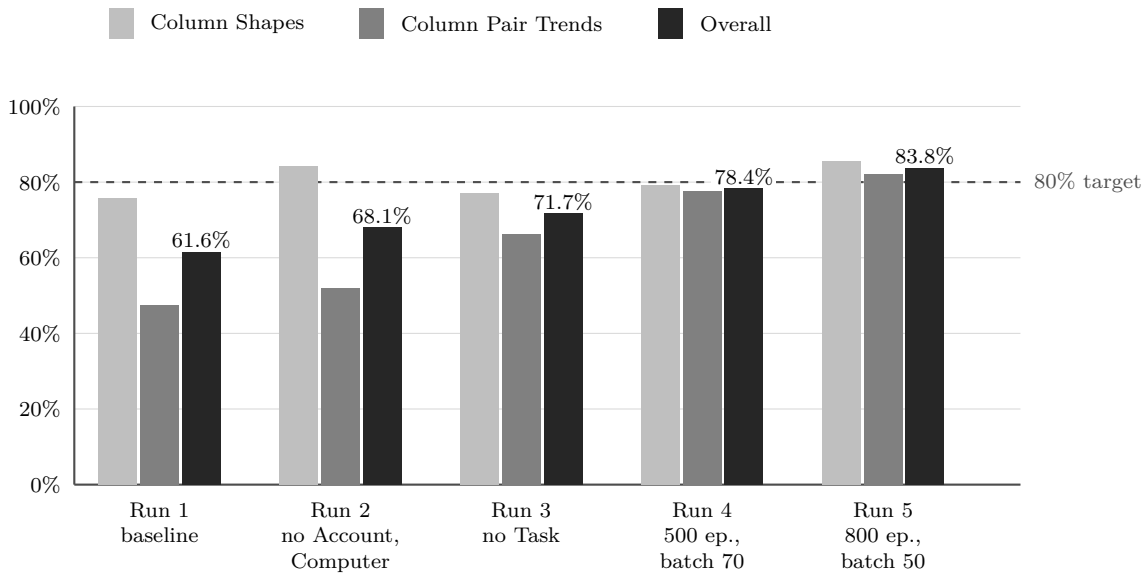


Figure 17: Quality score progression across the five training runs.

```
  GNU nano 8.7.1                                                    evaluate_quality.py
# Hamad Musaid - Project Mockingbird

import pandas as pd
from sdmetrics.reports.single_table import QualityReport
from sdv.metadata import SingleTableMetadata

# Load real (preprocessed) and synthetic data - using the same 6-column
real = pd.read_csv('preprocessed_variable.csv')
synth = pd.read_csv('synthetic_variable.csv')

# Ensure matching dtypes
categorical_cols = ['Account', 'AccountType', 'Computer', 'Task', 'EventID']
for col in categorical_cols:
    real[col] = real[col].astype(str)
    synth[col] = synth[col].astype(str)

# Auto-detect metadata (column types) from the real data
metadata = SingleTableMetadata()
metadata.detect_from_dataframe(real)

# Generate the Quality Report (Column Shapes + Column Pair Trends)
report = QualityReport()
report.generate(real, synth, metadata.to_dict())

print("\nOverall Quality Score:", report.get_score())
print("\nProperty breakdown:")
print(report.get_properties())
```

Figure 18: Run 1 script: baseline evaluation of all six columns with default training settings.

```
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~$ python3 evaluate_quality.py
/home/hamad/mockingbird-env/lib/python3.14/site-packages/sdmetrics/reports/single_table/__init__.py:12: FutureWarning: The single table quality report is deprecated. Please use the QualityReport from 'sdmetrics.reports' instead.
  from sdmetrics.reports.single_table.quality_report import QualityReport
Generating report ...

(1/2) Evaluating Column Shapes: |████████████████████████████████████████████████████████████| 6/6 [00:00<00:00, 79.88it/s]|
Column Shapes Score: 75.68%

(2/2) Evaluating Column Pair Trends: |██████████████████████████████████████████████████████| 15/15 [00:00<00:00, 248.04it/s]|
Column Pair Trends Score: 47.55%

Overall Score (Average): 61.61%


Overall Quality Score: 0.6161437159253946

Property breakdown:
             Property     Score
0       Column Shapes  0.756800
1  Column Pair Trends  0.475487
```

Figure 19: Run 1 result: overall 61.6% (shapes 75.7%, pair trends 47.6%).

```
  GNU nano 8.7.1                                                    evaluate_quality.py
# Hamad Musaid - Project Mockingbird


import pandas as pd
from sdmetrics.reports.single_table import QualityReport
from sdv.metadata import SingleTableMetadata

# Evaluate only the CTGAN-modeled columns (Account/Computer excluded by design)
model_cols = ['AccountType', 'Task', 'EventID', 'TimeGenerated_unix']

real = pd.read_csv('preprocessed_variable.csv')[model_cols]
synth = pd.read_csv('synthetic_variable.csv')[model_cols]

categorical_cols = ['AccountType', 'Task', 'EventID']
for col in categorical_cols:
    real[col] = real[col].astype(str)
    synth[col] = synth[col].astype(str)

metadata = SingleTableMetadata()
metadata.detect_from_dataframe(real)

report = QualityReport()
report.generate(real, synth, metadata.to_dict())

print("\nOverall Quality Score:", report.get_score())
print("\nProperty breakdown:")
print(report.get_properties())
```

Figure 20: `Account` and `Computer` are unique in all 697 rows, so the model cannot learn them -> Both fields were removed.



Figure 21: Run 2 script: `Account` and `Computer` removed from the model and the evaluation [Score = 68%].

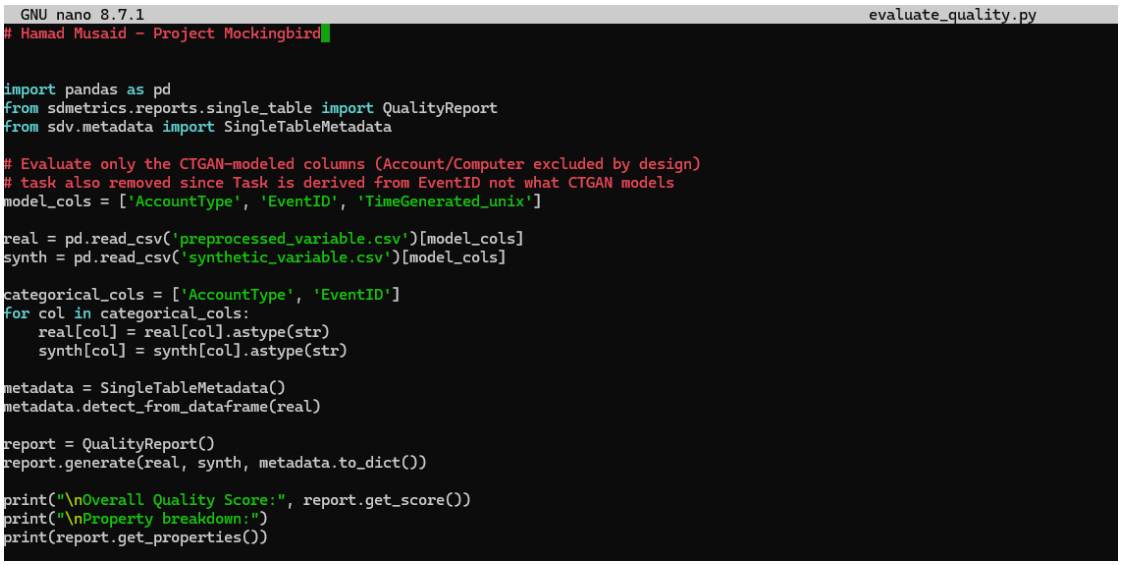


Figure 22: Script was updated to also remove `task` from the evaluation since it is derived from `EventID` not generated by the model.



Figure 23: Run 3 script: `Task` removed, overall score increased 72%



Figure 24: Run 4 result: overall 78.4% (shapes 79.1%, pair trends 77.7%) after retraining with 500 epochs and batch size 70.



Figure 25: Run 5 result: overall 83.8% (shapes 85.5%, pair trends 82.2%) after retraining with 800 epochs and batch size 50.

8.3 Final result

- The final model scored 83.8%, above the 80% target, so the Data fidelity criteria is met.

9 Sentinel Integration

9.1 Overview

- This stage pushed `synthetic_final.csv` from the virtual machine into a Microsoft Sentinel custom log table.
- The file is stored in an Azure Blob container over SFTP, and the records are pushed to the `GAN_CL_CL` table through the Azure Monitor Logs Ingestion API (Figure 26).

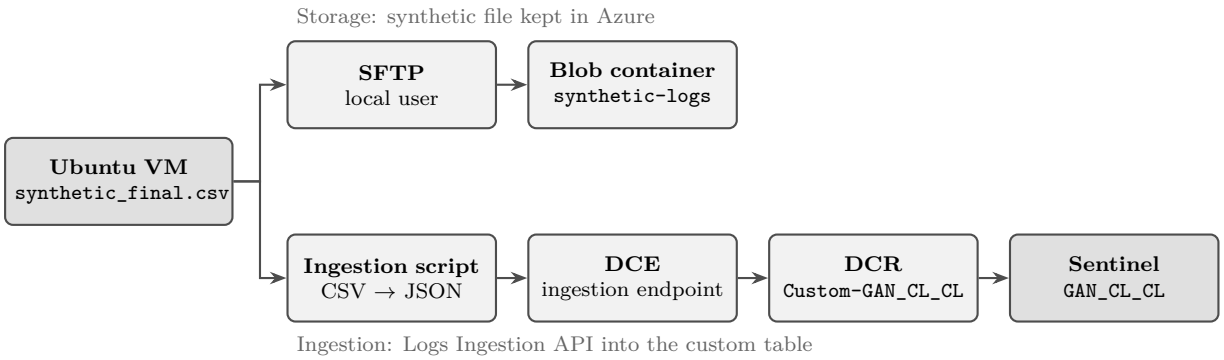


Figure 26: Integration architecture.

9.2 Storage account and SFTP upload

- **Storage account:** `ctgandataset`, with hierarchical namespace and SFTP enabled.
- **Container:** `synthetic-logs` holds the synthetic dataset.
- **Access:** a local SFTP user authenticated with an SSH password.
- **Upload:** the file was uploaded over SFTP from the Windows host, because the virtual machine had no network access at the time.

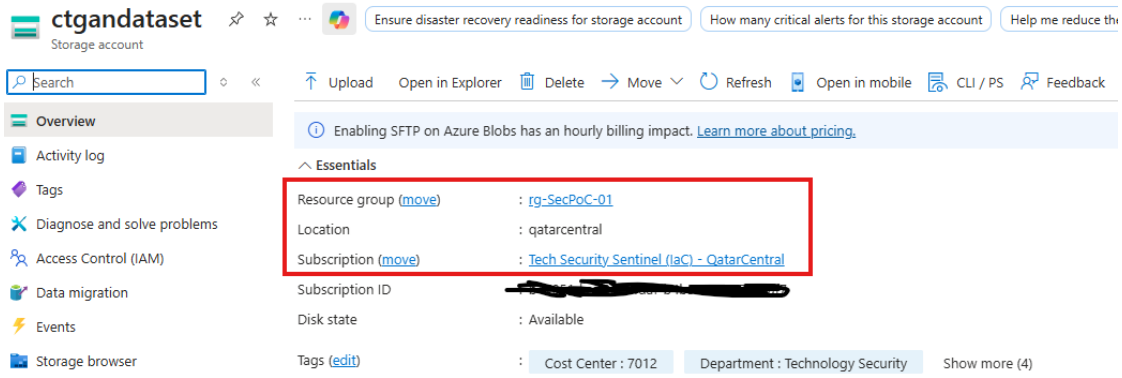


Figure 27: Storage account `ctgandataset`.

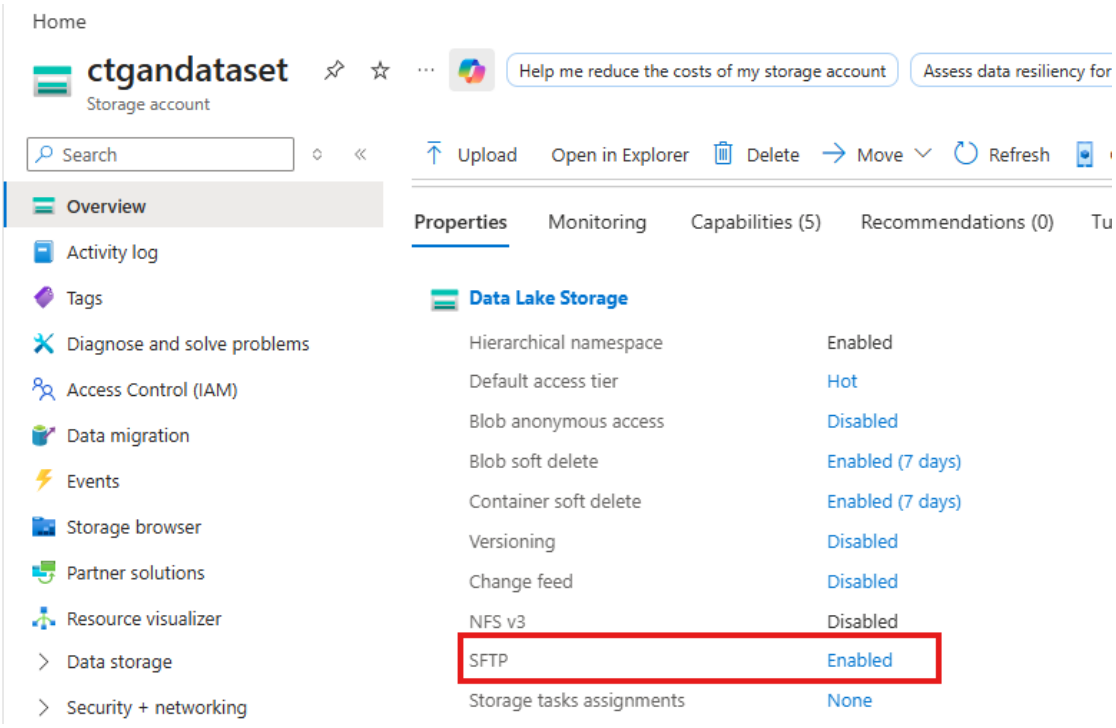


Figure 28: Hierarchical namespace and SFTP enabled on the storage account.

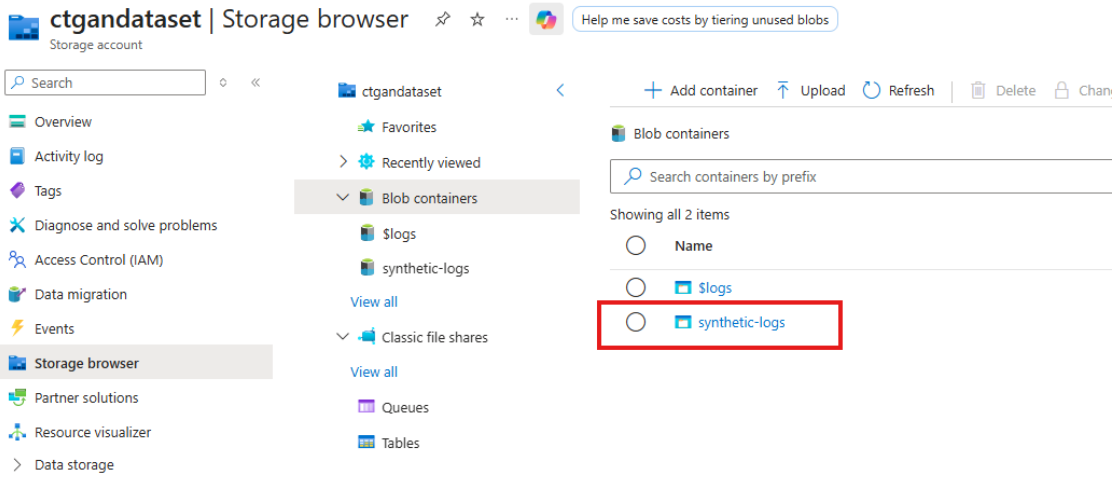


Figure 29: Blob container `synthetic-logs`.

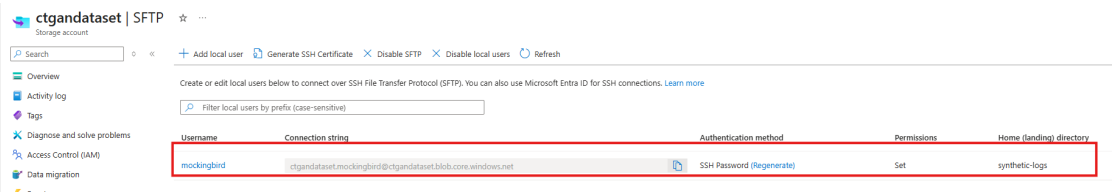


Figure 30: Local SFTP user created for the upload.

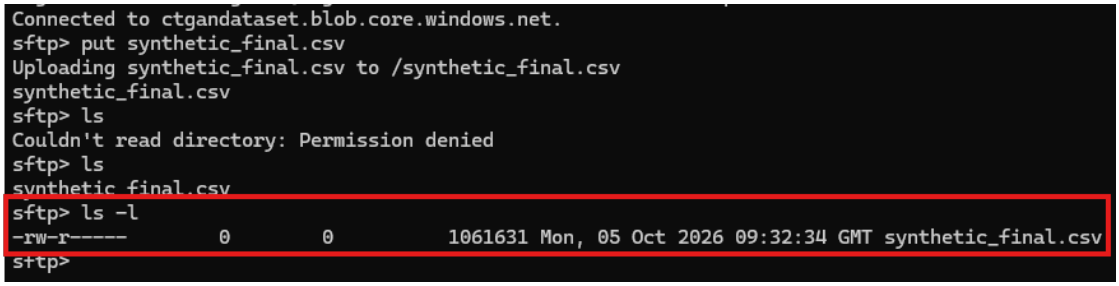


Figure 31: `synthetic_final.csv` uploaded to the container over SFTP.

9.3 Custom table and data collection rule

- **Custom table:** `GAN_CL_CL` in the Log Analytics workspace receives the synthetic records.
- **Data collection endpoint (DCE):** the URL the records are sent to.
- **Data collection rule (DCR):** declares the input stream `Custom-GAN_CL_CL`, its columns and types, and routes the records into the table.
- **Permission:** the *Monitoring Metrics Publisher* role on the DCR allows the ingestion script to send data.

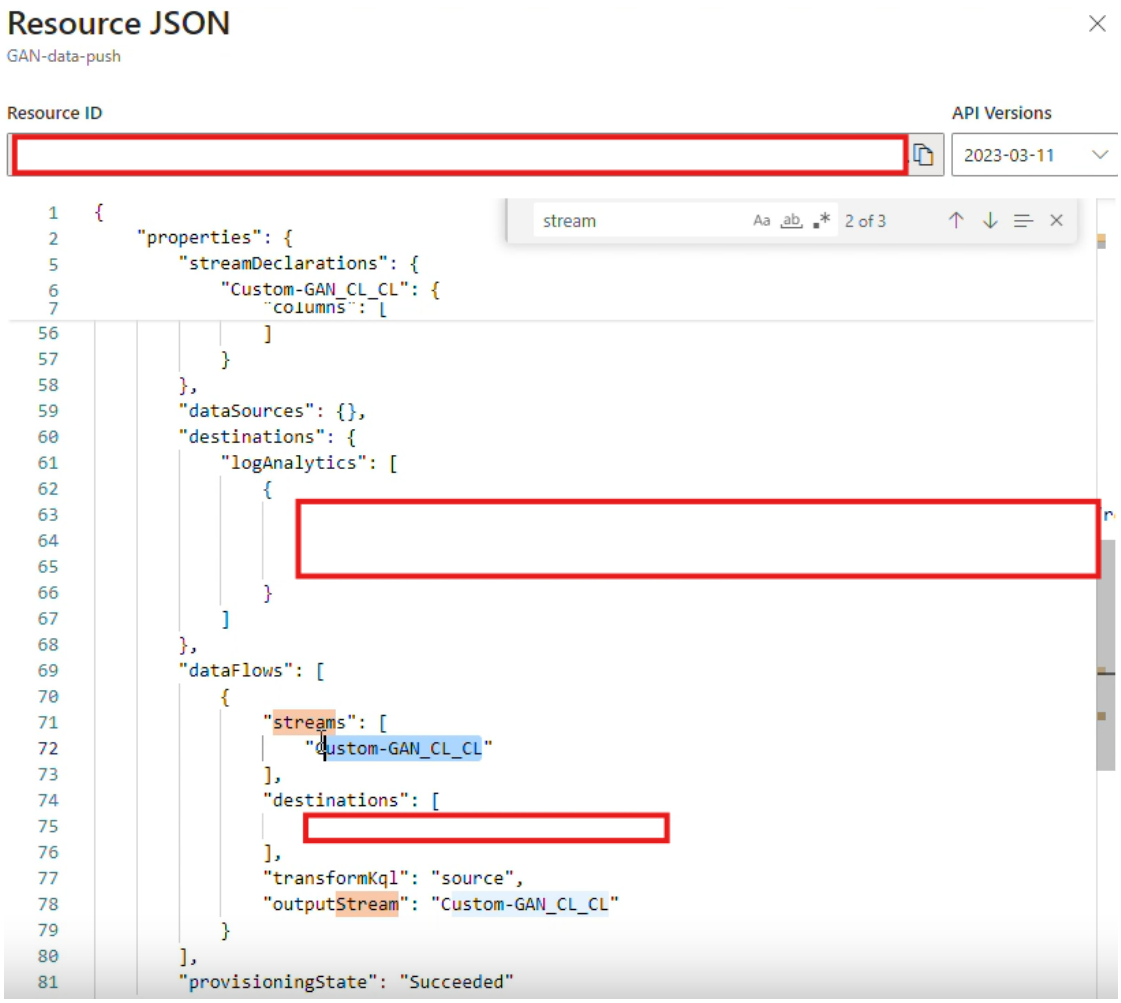


Figure 32: DCR JSON view: input stream `Custom-GAN_CL_CL` and its columns.

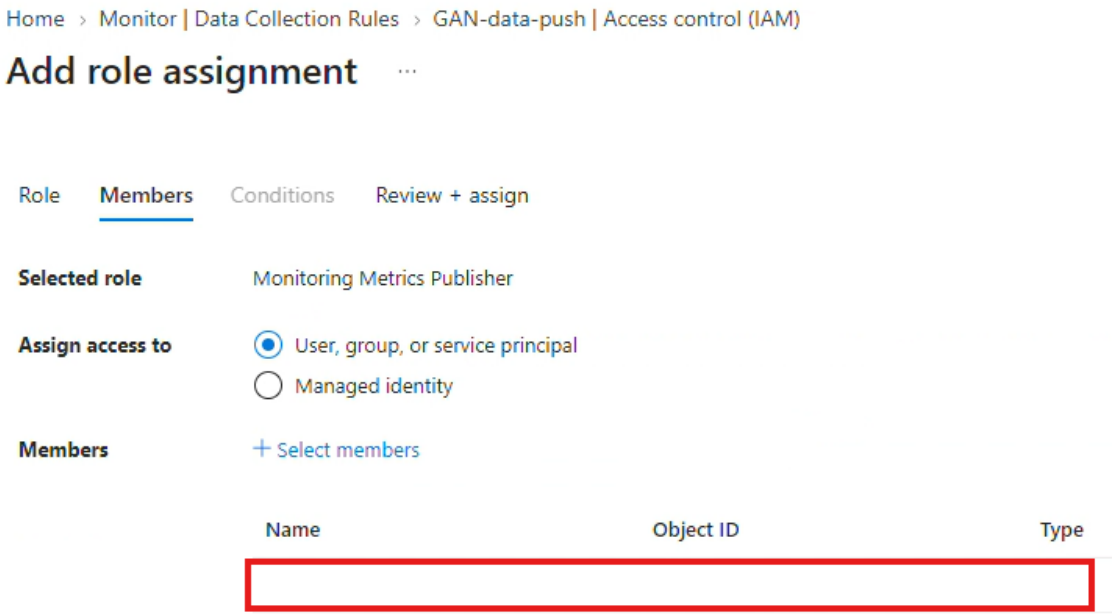


Figure 33: Monitoring Metrics Publisher role assigned on the DCR.

9.4 Ingestion script

The script `ingest_to_sentinel.py` is available in the project repository (https://github.com/upny-hm/mockingbird/blob/main/src/ingest_to_sentinel.py). It runs these steps:

- **Read:** load `synthetic_final.csv`. The final upload used the local copy on the virtual machine.
- **Drop `TenantId`:** it is a reserved column name in Log Analytics, which fills it automatically.
- **Set types:** `Task` and `EventID` as integers, all other columns as text.
- **Shift timestamps:** map the original time range into the last 24 hours, keeping order and spacing. Azure replaces timestamps older than two days, and analytic rules only search recent data.
- **Convert:** turn each row into a JSON record.
- **Authenticate:** sign in with a device code.
- **Upload:** send the records to the DCE and DCR. The Azure SDK splits them into chunks under the 1 MB request limit.

```
(mockingbird-env) hamad@hamad-VMware-Virtual-Platform:~/mockingbird/src$ python3 ingest_to_sentinel.py --local data/synthetic_final.csv
Reading local file: data/synthetic_final.csv
Loaded 5000 rows, 12 columns
Original time range: 2026-09-07 10:13:13+00:00 -> 2026-09-15 16:36:51+00:00
Shifted time range:  2026-10-07 07:29:11.564735+00:00 -> 2026-10-08 07:29:11.564735+00:00
Saved JSON: /home/hamad/mockingbird/src/data/synthetic_final.json

First record:
{
  "TimeGenerated": "2026-10-08T06:46:22.856581Z",
  "SourceSystem": "OpsManager",
  "Account": "Account434",
  "AccountType": "Unknown",
  "Computer": "10.168.5.38",
  "EventSourceName": "Microsoft-Windows-Security-Auditing",
  "Channel": "Security",
  "Task": 14339,
  "Level": "0.0",
  "EventID": 4771,
  "Activity": "4771 - Kerberos pre-authentication failed."
}
Sending 5000 records to Custom-GAN_CL_CL ...
To sign in, use a web browser to open the page https://login.microsoft.com/device and enter the code [redacted] to authenticate.
Done: 5000 sent, 0 failed.
```

Figure 34: Ingestion run: 5,000 records sent, 0 failed.

9.5 Verification

- A query on `GAN_CL_CL` counts the records and checks that every column is populated (Figure 35).
- The table holds 5,009 records: the 5,000 synthetic rows plus 10 rows from an initial test upload, which remain in the table.
- Every column is filled in every row, so the data was ingested without parsing errors and the Integration criterion is met.

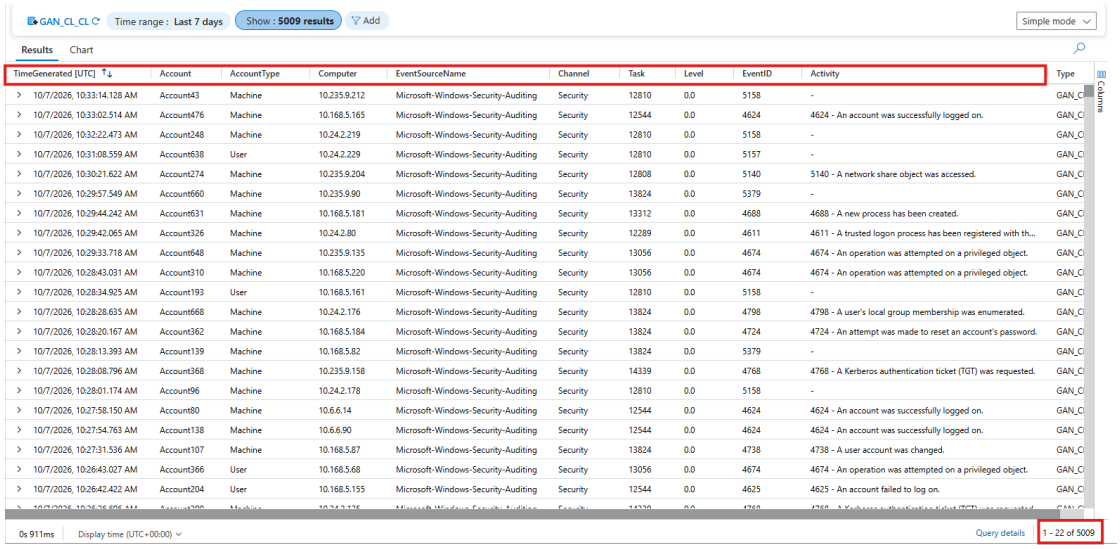


Figure 35: Records in `GAN_CL_CL` with all columns populated.

10 Validation

10.1 Approach

- Organizational analytic rules and incidents are confidential and are not reproduced in this report. Instead validation was carried out at he Query level: a password spray detection query was run against `GAN_CL_CL` in the Sentinel workspace.
- Every event the query evaluates was generated by CTGAN. No attack events were added by hand.

10.2 Password spray detection

A password spray tries one or two common passwords against many accounts, so it shows up as many accounts failing to log on within a short time. The query looks for three signs of this, in one-hour windows:

- **High volume of failed logons:** `FailedAttempts`, and `SpikeRatio` against the normal hourly rate (the median over the last day = 2 failed attmpts per account).
- **Failed logons against active users:** `ActiveTargeted`, the accounts that also logged on successfully in the same period.
- **Failed logons for nonexistent accounts:** `UnknownTargeted`, accounts that never appear outside failed logons.

A window is flagged when at least 20 accounts fail and failures are at least 10 times the normal rate. Severity is High if nonexistent accounts are targeted or failures are at least 50 times the normal rate, and Medium otherwise.

```
let lookback      = 1d;      // rule "Lookup data from last"
let window        = 1h;      // "brief period"
let min_accounts  = 20;      // distinct accounts failing in the window
let min_spike     = 10.0;    // failures vs the normal hourly rate
let fail_ids      = dynamic([4625, 4771]);   // failed logon, Kerberos pre-auth failed
let success_ids   = dynamic([4624, 4768]);   // successful logon, Kerberos TGT issued
let events   = GAN_CL_CL | where TimeGenerated > ago(lookback);
let failures = events | where EventID in (fail_ids);
// Accounts that are active: they logged on successfully in the lookback period
let active_accounts = events | where EventID in (success_ids) | distinct Account | extend IsActive = true;
// Accounts that exist: they appear in any event other than a failure
let known_accounts  = events | where EventID !in (fail_ids) | distinct Account | extend IsKnown = true;
// Normal number of failed logons per hour (median of the lookback period)
let baseline = toscalar(
    failures
    | summarize c = count() by bin(TimeGenerated, window)
    | summarize percentile(c, 50));
failures
| join kind=leftouter active_accounts on Account
| join kind=leftouter known_accounts on Account
| summarize
    StartTime        = min(TimeGenerated),
    EndTime          = max(TimeGenerated),
    FailedAttempts   = count(),
    TargetedAccounts = dcount(Account),
    SourceIPs        = dcount(Computer),
    ActiveTargeted   = dcountif(Account, IsActive == true),
    UnknownTargeted  = dcountif(Account, isnull(IsKnown)),
    SampleAccounts   = make_set(Account, 20),
    SampleSourceIPs  = make_set(Computer, 20)
    by Window = bin(TimeGenerated, window)
| extend SpikeRatio = round(todouble(FailedAttempts) / max_of(todouble(baseline), 1.0), 1)
| where TargetedAccounts >= min_accounts and SpikeRatio >= min_spike
| extend Severity = iff(UnknownTargeted > 0 or SpikeRatio >= 50, "High", "Medium")
| project StartTime, EndTime, FailedAttempts, SpikeRatio, TargetedAccounts, ActiveTargeted,
          UnknownTargeted, SourceIPs, Severity, SampleAccounts, SampleSourceIPs
```

Figure 36: Password spray detection query in the Sentinel Logs editor.

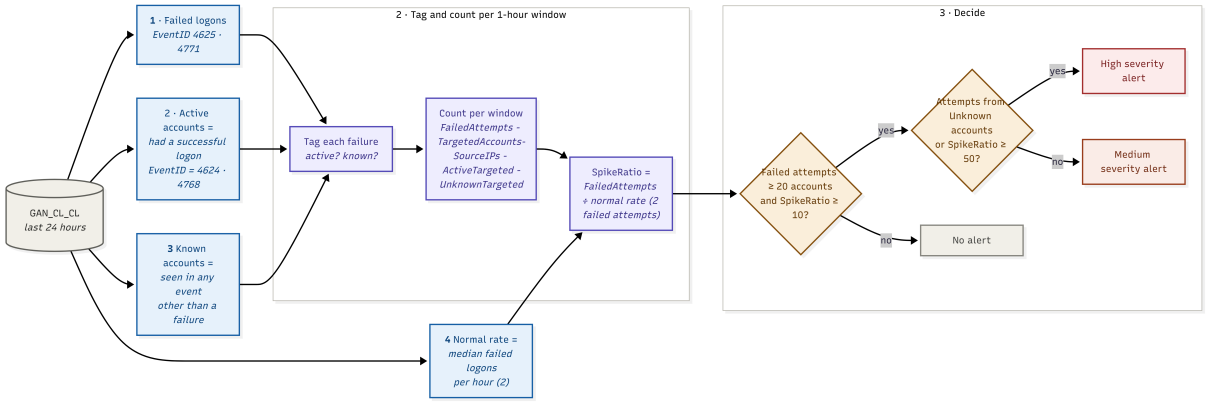


Figure 38: KQl query logic

10.3 Results

The query returned two detections (Figure 37). The full output is available in the project repository. (https://github.com/upny-hm/mockingbird/blob/main/src/data/KQL_query_output.csv)

StartTime [UTC]	EndTime [UTC]	FailedAttempts	SpikeRatio	TargetedAccounts	ActiveTargeted	UnknownTargeted	SourceIPs	Severity	SampleAccounts	SampleSourceIPs
10/8/2026, 7:00:06.309 AM	10/8/2026, 7:22:39.859 AM	43	21.5	42	10	0	42	Medium	["Account340","Account1	["10.168.5.113","10.24.2.103","10.24.2.172"
10/8/2026, 6:23:15.192 AM	10/8/2026, 6:59:14.654 AM	123	61.5	114	29	0	111	High	["Account156","Account3	["10.168.5.144","10.168.5.96","10.235.9.82"

Figure 37: Query output: two flagged one-hour windows.

- **One attempt, two time windows:** the two detections are back to back. **6:23:16 AM - 6:59:14 AM** recorded 143 failed login attempts and a 61.6 spike ratio which set off the rule. **7:00:06 AM - 7:22:39 AM** also saw 43 failed login attempts and a spike ration of 21.6 Spike Ratio which also set off the rule.
- **Distributed pattern:** failures came from more than 100 different source IPs, indicating a password spray attempt, where attempts are spread across many sources to stay below normal failed login thresholds.
- **Active users targeted:** 29 and 10 of the targeted accounts had logged on successfully in the same period.
- **No nonexistent accounts:** `UnknownTargeted` is 0, because CTGAN can only reuse account names it saw during training.
- **Cause of the burst:** most events fall into a short period, because CTGAN reproduced the busy period of the original data and the timestamps were mapped into the last 24 hours. The share of failed logons inside the burst is the same as across the whole dataset, so the detection reflects a surge in volume rather than an attack-specific pattern.

11 Conclusion

Project Mockingbird showed that a CTGAN model trained on a small, sanitized sample of Windows security events can generate realistic synthetic logs, deliver them into Microsoft Sentinel, and trigger detection logic, without exposing real user data. All four success criteria were met:

- **Data fidelity:** the final model scored 83.8% on the SDMetrics quality report, above the 80% target (Section 8).
- **Formatting:** IP addresses, usernames and timestamps in the synthetic data matched the original formats (Section 7).
- **Integration:** 5,000 synthetic records were ingested into the `GAN_CL_CL` custom table with every column populated (Section 9).
- **Validation:** a password spray detection query flagged the CTGAN-generated data in Sentinel (Section 10).

The project also showed the limits of a row-based generator. CTGAN learns what individual events look like, but not patterns that span many events, so it cannot produce targeted attack sequences or invent new account names on its own. Training on labeled attack examples, or using a generator that models event sequences, would let future work produce specific attack scenarios on demand. Even in its current form, the pipeline gives the SOC a repeatable, privacy-safe way to generate test data and check detection logic.